CLASE 5 - Sistemas Digitales

Semana 5: Temporizadores de hardware en el ESP32

1. Desbordamiento de variables (Overflow)

El desbordamiento ocurre cuando una variable intenta almacenar un valor que supera el rango permitido por su tipo de dato.

Por ejemplo, `uint8_t` es un tipo de dato entero sin signo que utiliza 8 bits y permite almacenar valores entre 0 y 255.

```
uint8_t x = 200;
uint8_t y = 100;

uint8_t resultado = x + y;
```

La suma es:

$$200 + 100 = 300$$

Pero `uint8_t` no puede almacenar 300. Al superar el límite de 255, el resultado vuelve al inicio del rango:

$$300 - 256 = 44$$

Por lo tanto, el resultado almacenado será `44`.

Esto sucede porque una variable de 8 bits sin signo trabaja con 256 valores posibles, desde 0 hasta 255.

Nota: en C/C++, la suma se realiza primero con promoción de enteros y después el resultado se convierte a `uint8_t` al almacenarse.

2. Temporizadores de hardware (Hardware Timers)

Un temporizador es un periférico, es decir, un circuito interno del microcontrolador que realiza una función específica y reduce la carga de trabajo de la CPU.

Es un circuito que cuenta pulsos de reloj. Como cada pulso tiene una duración conocida, permite medir intervalos de tiempo con precisión.

La CPU configura el temporizador y este puede continuar contando de manera independiente. Cuando alcanza un valor determinado, puede generar una alarma o interrupción.

Diferencia entre `delay()` y `millis()`

`**delay()**`

Detiene temporalmente la ejecución de la tarea durante el tiempo indicado.

```
delay(1000);
```

En este caso se espera un segundo antes de continuar con las siguientes instrucciones de esa tarea.

`**millis()**`

Devuelve el tiempo transcurrido desde que comenzó el programa, expresado en milisegundos.

Permite comprobar si ha pasado determinado tiempo sin detener completamente la ejecución del código.

```
unsigned long anterior = 0;

void loop() {
    unsigned long actual = millis();

    if (actual - anterior >= 1000) {
        anterior = actual;
        // Ejecutar una acción cada segundo
    }
}
```

Mientras se espera, el programa puede seguir realizando otras operaciones.

Característica	**`delay()`**	**`millis()`**
Funcionamiento	Espera durante un tiempo.	Consulta el tiempo transcurrido.
Bloqueo	Bloquea la tarea que lo ejecuta.	No bloquea por sí mismo.
Uso	Pausas sencillas.	Control de varias acciones temporizadas.

Temporizador como organizador: se puede comparar con un reloj independiente que cuenta el tiempo y avisa cuando se cumple una condición. La CPU no necesita contar manualmente cada pulso; configura el temporizador y puede dedicarse a otras tareas.

3. El contador del temporizador

El contador es un registro interno del temporizador donde se almacena el valor actual de la cuenta.

Su funcionamiento básico es:

1. La CPU configura el temporizador.
2. Se establece el valor inicial y las condiciones de funcionamiento.
3. La CPU habilita el temporizador.
4. El contador comienza a incrementarse automáticamente.
5. Cuando alcanza el valor de alarma, puede generar un evento.
6. La CPU puede detenerlo, reiniciarlo o modificar su configuración.

El incremento de uno significa que el contador suma una unidad por cada tick o pulso de reloj que recibe.

Por ejemplo:

```
0 → 1 → 2 → 3 → 4 → 5 → ...
```

La CPU no necesita ejecutar una instrucción para incrementar cada número: el circuito del temporizador realiza el conteo.

4. El reloj del temporizador

Para medir el tiempo, el temporizador necesita una señal de reloj.

4.1. Reloj

Es una señal eléctrica periódica que cambia entre dos niveles:

- Nivel bajo (0).
- Nivel alto (1).

Cada ciclo completo, formado por una subida y una bajada de la señal, representa un período de reloj.

La duración de ese período se calcula mediante:

$$T_s = \frac{1}{F}$$

Donde:

- T_s: período de la señal, en segundos.
- F: frecuencia, en hercios (Hz).

4.2. Reloj APB

APB significa *Advanced Peripheral Bus*. Es un bus utilizado para conectar y comunicar la CPU con determinados periféricos del ESP32.

En el ESP32 clásico, el reloj APB normalmente funciona a 80 MHz y sirve como fuente de reloj para los temporizadores tradicionales.

Esto significa que puede proporcionar:

$$80, \mathrm{MHz} = 80,000,000 \ \mathrm{Hz}$$

Es decir, 80 millones de ciclos por segundo.

4.3. Duración de cada pulso

Si el reloj trabaja a 80 MHz:

$$T_s = \frac{1}{80,000,000}$$

$$T_s = 0.0000000125 \ \mathrm{s}$$

Equivale a:

$$T_s = 12.5 \ \mathrm{ns}$$

Por tanto, cada ciclo de reloj dura 12.5 nanosegundos.

5. Prescaler (divisor)

El prescaler es un divisor de frecuencia que permite reducir la cantidad de pulsos que recibe el contador del temporizador.

En lugar de contar los 80 millones de pulsos por segundo directamente, se divide la frecuencia para trabajar con una más cómoda.

La relación es:

$$F_{\mathrm{timer}} = \frac{F_{\mathrm{reloj}}}{N}$$

Donde:

- F_{timer}: frecuencia con la que incrementa el contador.
- F_{reloj}: frecuencia de entrada, por ejemplo 80 MHz.
- N: divisor o prescaler.

Ejemplo de funcionamiento

Si el reloj original tiene una frecuencia de 80 MHz y se utiliza un divisor de 80:

$$F_{\mathrm{timer}} = \frac{80,000,000}{80}$$

$$F_{\mathrm{timer}} = 1,000,000 \ \mathrm{Hz}$$

El contador recibe un millón de ticks por segundo.

Esto permite que cada incremento represente un microsegundo.

Interpretación de las gráficas

Las gráficas de reloj representan una señal que alterna entre nivel bajo y nivel alto.

- Una frecuencia alta significa que hay muchos ciclos en poco tiempo.
- Una frecuencia baja significa que hay menos ciclos en el mismo intervalo.
- El prescaler divide la frecuencia, por lo que el contador recibe menos pulsos.

Por ejemplo, si se divide entre 3, el contador recibe un tick por cada tres ciclos de la señal original.

La idea es elegir una unidad de tiempo cómoda para trabajar y medir intervalos mediante el número de ticks acumulados.

6. Resolución del temporizador

La resolución es la duración de un tick del contador. Representa el tiempo más pequeño que se puede medir con ese incremento.

Se calcula mediante:

$$R = \frac{1}{F_{\text{timer}}}$$

Donde F_{timer} es la frecuencia que llega al contador después de aplicar el prescaler.

Por ejemplo, si la frecuencia del temporizador es 1 MHz:

$$R = \frac{1}{1,000,000}$$

$$R = 0.000001 \text{ s} = 1, \mu s$$

Cada tick representa un microsegundo.

Relación entre reloj, prescaler y resolución:

1. El reloj proporciona los pulsos.
2. El prescaler divide la frecuencia.
3. El contador recibe los ticks.
4. La resolución indica cuánto tiempo representa cada tick.

7. Alarma y cálculo del tiempo

La alarma es un valor que se configura en el temporizador para determinar cuándo debe generarse un evento.

El contador avanza hasta alcanzar ese valor. Cuando llega a la alarma, puede generar una interrupción o activar una acción configurada.

Se puede entender mediante esta comparación:

- **Resolución:** duración de cada peldaño.
- **Alarma:** cantidad de peldaños que debe recorrer el contador.
- **Tiempo:** duración total del recorrido.

La fórmula principal es:

$$T = \text{Alarma} \times R$$

También se puede calcular la cantidad de ticks necesarios:

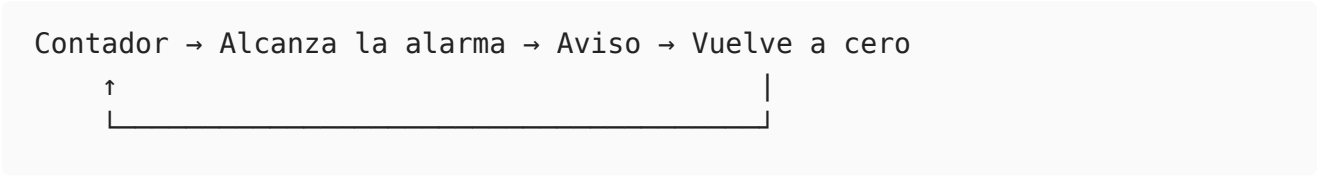
$$\text{Alarma} = T \times F_{\text{timer}}$$

7.1. Autorrecarga (Auto-reload)

La autorrecarga permite que el contador vuelva automáticamente a su valor inicial cuando alcanza el valor de alarma.

Su funcionamiento es:

1. El contador comienza a incrementarse.
2. Alcanza el valor de alarma.
3. Se genera el aviso o evento configurado.
4. El contador vuelve a cero.
5. Comienza nuevamente el conteo.



Esto permite generar eventos periódicos sin tener que iniciar manualmente el temporizador después de cada alarma.

7.2. Desbordamiento del contador

El desbordamiento ocurre cuando el contador alcanza su valor máximo y vuelve a cero, debido a que no puede representar un número mayor.

A diferencia de la alarma, el desbordamiento depende del límite de bits del contador.

- La alarma se configura para generar un evento en un valor elegido.
- El desbordamiento ocurre cuando se supera la capacidad máxima del contador.
- Puede aprovecharse intencionalmente para realizar conteos cíclicos.

Por ejemplo, en un contador de 8 bits:

```
... → 253 → 254 → 255 → 0 → 1 → ...
```

El contador vuelve a cero al superar 255, aunque no haya alcanzado el valor de alarma configurado.

7.3. Límite del contador

La capacidad máxima de un contador depende de la cantidad de bits que utiliza.

Para un contador de n bits sin signo:

$$\text{Máximo} = 2^n - 1$$

Ejemplos:

Bits	Valor máximo
8 bits	255
16 bits	65 535
64 bits	18 446 744 073 709 551 615

El ESP32 clásico utiliza temporizadores generales con contadores de 64 bits. Por eso, su capacidad de conteo es muy grande y el desbordamiento tarda mucho en producirse, dependiendo de la frecuencia de los ticks.

8. Ejercicios de temporizadores

Ejercicio 1

Datos:

- Frecuencia del reloj: 80 MHz.
- Frecuencia deseada del temporizador: 1 MHz.
- Tiempo deseado: 0.5 segundos.

Paso 1. Calcular la resolución

$$R = \frac{1}{1,000,000}$$

$$R = 0.000001 \text{ s} = 1, \mu s$$

Cada tick representa un microsegundo.

Paso 2. Calcular la alarma

$$A = T \times F_{\text{timer}}$$

$$A = 0.5 \times 1,000,000$$

$$A = 500,000 \text{ ticks}$$

Paso 3. Comprobar el tiempo

$$T = A \times R$$

$$T = 500,000 \times 0.000001$$

$$T = 0.5 \text{ s}$$

Paso 4. Calcular el prescaler

$$N = \frac{F_{\text{reloj}}}{F_{\text{timer}}}$$

$$N = \frac{80,000,000}{1,000,000}$$

$$N = 80$$

Resultado: se utiliza un divisor de 80, una resolución de 1 microsegundo y una alarma de 500 000 ticks para obtener medio segundo.

Ejercicio 2

Datos:

- Frecuencia deseada del temporizador: 10 000 Hz.
- Alarma: 2 500 ticks.
- Frecuencia del reloj: 80 MHz.

Paso 1. Calcular la resolución

$$R = \frac{1}{10,000}$$

$$R = 0.0001 \text{ s} = 0.1 \text{ ms}$$

Cada tick representa 0.1 milisegundos.

Paso 2. Calcular el tiempo

$$T = A \times R$$

$$T = 2,500 \times 0.0001$$

$$T = 0.25 \text{ s}$$

Paso 3. Calcular el prescaler

$$N = \frac{80,000,000}{10,000}$$

$$N = 8,000$$

Resultado: se necesita un divisor de 8 000, una resolución de 0.1 milisegundos y una alarma de 2 500 ticks para obtener 0.25 segundos.

9. Temporizadores del ESP32

El ESP32 clásico dispone de cuatro temporizadores generales de hardware, organizados en dos grupos:

Grupo	Temporizadores
TIMG0	Timer 0 y Timer 1
TIMG1	Timer 0 y Timer 1

Cada temporizador puede configurarse de forma independiente para contar pulsos, establecer una alarma y generar interrupciones.

Esta organización permite utilizar varios temporizadores para diferentes tareas, por ejemplo:

- Medir intervalos de tiempo.
- Generar eventos periódicos.
- Controlar la frecuencia de ciertas acciones.
- Ejecutar tareas relacionadas con sensores o actuadores.

10. API para controlar los temporizadores

Una API (*Application Programming Interface*) proporciona funciones ya implementadas que permiten utilizar los recursos del microcontrolador sin tener que configurar manualmente todos sus registros.

En lugar de modificar directamente los registros del temporizador, el programa puede llamar a las funciones de la API para configurarlo.

El proceso general es:

```
Programa
   ↓
Llama a una función de la API
   ↓
La API configura el prescaler y la alarma
   ↓
Se modifican los registros del temporizador
```

```
    ↓
Se habilita el contador
```

La API facilita el trabajo porque se encarga de realizar las configuraciones necesarias mediante las funciones disponibles en el framework.

En ESP-IDF existen controladores para los temporizadores. Las funciones concretas dependen de la versión del framework y del periférico utilizado.

11. Interrupciones

Una interrupción es una señal que provoca que el procesador suspenda temporalmente la ejecución normal para atender un evento que necesita una respuesta.

Cuando ocurre una interrupción:

1. El procesador pausa temporalmente el código que estaba ejecutando.
2. Ejecuta una rutina encargada de atender el evento.
3. Una vez atendido, regresa al punto donde había interrumpido la ejecución.

Las interrupciones permiten responder a eventos sin tener que consultar constantemente si algo ocurrió.

11.1. Interrupciones internas y externas

Interrupciones internas

Son generadas por eventos dentro del propio microcontrolador o sus periféricos.

Ejemplos:

- Un temporizador alcanza el valor de alarma.
- Se completa una operación de comunicación.
- Se produce un evento de un periférico interno.

Pueden configurarse para ocurrir periódicamente, por ejemplo, cada 500 milisegundos.

Interrupciones externas

Se producen por señales o eventos provenientes del exterior del microcontrolador.

Ejemplos:

- Pulsar un botón.
- Detectar un cambio de nivel lógico en un GPIO.
- Recibir una señal de un sensor externo.

Su momento de ocurrencia puede ser impredecible, porque depende de cuándo sucede el evento externo.

11.2. ISR (Interrupt Service Routine)

ISR significa *Interrupt Service Routine* o rutina de servicio de interrupción.

Es la función que se ejecuta cuando ocurre una interrupción. Su propósito es atender rápidamente el evento que la provocó.

Por ejemplo, si un temporizador genera una interrupción, la ISR puede marcar que ha transcurrido un intervalo de tiempo.

Características:

- Se ejecuta como respuesta a una interrupción.
- Interrumpe temporalmente el flujo normal del programa.
- Debe realizar operaciones breves.
- Al terminar, permite continuar con la ejecución normal.

Importante: una ISR no debe realizar operaciones largas ni utilizar funciones bloqueantes como `delay()`. Conviene evitar tareas pesadas y dejar el procesamiento más complejo para el programa principal o una tarea de FreeRTOS.

11.3. Callback

Un callback es una función escrita por el programador que no se llama directamente desde el flujo principal, sino que es invocada por otra función o componente cuando ocurre una condición determinada.

Por ejemplo, al configurar un temporizador, se puede indicar qué función debe ejecutarse cuando se active su alarma.

La relación es:

- El programador escribe la función callback.
- La API recibe o registra esa función.
- Cuando ocurre el evento correspondiente, el sistema la invoca.

En algunos controladores, el callback se ejecuta directamente como parte de una interrupción; en otros, puede ejecutarse desde una tarea. Depende de la API utilizada.

Por eso, es importante saber en qué contexto se ejecuta el callback y qué operaciones se pueden realizar dentro de él.